

NAME

kh – two-player tactical space combat game command interface

SYNOPSIS

```
kh
kh --help
kh --version
kh --port PORT
kh --dbusr USER
    --dbpass PASSWORD
    --dbname DATABASE
    --dbhost HOST
kh --monitor
kh --log FILE
```

DESCRIPTION

Kepler's Horizon is a two-player tactical game of space combat where players build individualized ships, move them across a hex-based star map, and resolve combat without dice. The objective is to occupy opponent base star hexes and accumulate victory points.

This manual describes all available commands in the Kepler's Horizon command language as defined by the game grammar.

OPTIONS**Generic Program Information**

--help Output a usage message and exit.

-v, --version
Output the version number of **kh** and exit.

Database Connection

-H, --dbhost *HOST*
The hostname to reach for the database. Default is **127.0.0.1**

-u, --dbusr *USER*
Access the underlying database as *USER*

-d, --dbname *DB_NAME*
The name of the underlying database.

-p, --dbpass *PASSWORD*
The password for *USER*.

Database Connection

-P, --port *PORT*
The port number the server listens to for requests via REST. The default is 8080.

Logging

-L, --log *FILE*
Logging from the server written to *FILE*.

Monitor

-M, --monitor
Put the server into monitor mode. The server will accept commands from terminal prompt.

GAME OVERVIEW**Map Components****StarHex**

A map hex containing a large or small star (graph node)

BaseStarHex

A map hex containing a large star (6 total: 3 per player)

SpaceHex

A map hex without a star or warpline

WarpLine

Bi-directional edges connecting stars for instantaneous travel

Ship Types**WarpShip (W#)**

Ships capable of interstellar travel via warplines

SystemShip (S##)

Ships that cannot travel between stars unless carried by WarpShips

GAME SEQUENCE

Each player-turn consists of the following phases in order:

1. Count Victory Points
2. Build Ships (receive BP at start of each turn)
3. Movement
4. Resolve Combat
5. SystemShip Pick and Drop
6. End of Turn

CONCEPTS**Starting a Game**

Games are started through the web interface, not via command line:

1. Portal

Login or create account at the web portal.

2. Lobby

View open games or create a new one.

3. Join Game

Join a game and take a seat (A or B). When both seats are filled, either player can start the game.

4. Module Selection

The game host selects the universe module:

Kepler's Horizon

The original frontier universe - humanity's expansion into the stars. Contains 28 star systems across Alpha and Beta territories.

(Additional Modules)

Custom modules can be created with unique star maps, populations, species, and resources. See the wiki for the Module Authoring guide.

MODULES

A **Module** defines a complete game universe. All milieu data (star systems, planets, species, populations, resources) is keyed by module_id.

The game engine is module-agnostic; only content varies between modules. Players select a module when creating a game.

See the wiki for the Module Authoring guide.

COMMANDS

Commands are case-insensitive. Aliases are shown after commas.

Turn Management Commands

next, n Advance the active player to the next phase.

Syntax: next

Example:

> next

Phase advanced to: Movement

done, ZZ

Complete current player's turn and advance to opponent's first phase.

Syntax: done

Example:

> done

COMMAND: Your turn has ended. Standing down.

Information Commands

status Show game status and important records (current phase, turn number, BP, VP).

Syntax: status

fleet Show manifest of your fleet (all ships and their current attributes).

Syntax: fleet

Example:

> fleet

Your Fleet (Player A):

W1 The Vanguard [PD:5 B:3 S:2 T:1 M:6] at ARVEN

W2 The Sentinel [PD:4 B:2 S:3 T:0 M:0] at BELIX

crt Display the Combat Results Table

hex *location*

Show location and spatial information about the specified hex

online Show which players are online and their connection times

stats Display statistics about the game, your fleet, and opponent

fleet Show manifest of your fleet (all ships and their current attributes)

system *name*

Show environmental information about the specified star system

galaxy Complete overview of all systems and all ships in the game

score Show current game status: whose turn, phase, credits, tech level, and victory points

Economy Commands

extract *scan*

Scan current system for harvestable resources. Requires ship with LRS (Long Range Scanner).

extract *ship resource*

Extract resource from current system into ship's cargo hold. Requires TB (Transporter Beam) or DR (Drones).

Resources: FERROUS, RARE_EARTH, RADIOACTIVE, CRYSTALLINE, VOLATILE, WATER, ORGANIC, EXOTIC

market Display current market prices with trend indicators (↑ rising, → stable, ↓ falling).

Prices fluctuate each round based on supply/demand and market forces.

market resource

Show price history for the specified resource over recent turns.

trade list

Show base trade prices for all resources.

trade buy resource quantity

Purchase resources at current market price, deducting credits.

trade sell resource quantity

Sell resources from cargo at 75% of market price.

trade transfer from_ship to_ship resource quantity

Move resources between ships at the same location.

fabricate list

Show available manufacturing recipes and their costs.

fabricate recipe quantity

Manufacture items from raw materials in cargo. Recipes include:

missiles

Basic missiles (4 per batch). Cost: 2 Fe, 1 Rd, 1 Vo. Time: 1 round.

tubes

Tube upgrade (+1 capacity). Cost: 5 Fe, 3 Re, 2 Cr. Time: 3 rounds.

beams

Beam upgrade (+1 rating). Cost: 8 Fe, 4 Re, 3 Cr. Time: 3 rounds.

screens

Screen upgrade (+1 rating). Cost: 6 Fe, 2 Re, 4 Cr. Time: 3 rounds.

tech

Tech research (+1 level). Cost: 10 Re, 5 Cr, 2 Ex. Time: 5 rounds.

salvage ship

Salvage wreckage at anomaly sites like The Graveyard (SYDRA). Requires Drones (DR). Random outcome: 90% chance of recovering 2-5 units of random resources, 10% chance of hazard encounter dealing hull damage.

Build Commands

build, b

Show current build state (active drafts, available BP)

build new code name

bn code name Create a new ship draft with the specified code (e.g., W1, S01) and name

build drafts

bd List all pending build drafts

build drafts ship_code

bd ship_code Show detailed attributes of a specific draft ship

build set attributes, bs attributes

Set attributes on the current draft ship. Attributes are specified as:

pd=N or **d=N**

Power/Drive strength (1 BP per unit, provides movement and combat power)

b=N

Beam weapon strength (1 BP per unit, max power for beam projector)

s=N

Screen strength (1 BP per unit, absorbs hits equal to power + tech level)

t=N

Number of missile tubes (1 BP per tube, requires 1 PD to fire)

m=N

Number of missiles (1 BP per 3 missiles, each does 2 + tech level hits)

sr=N SystemShip racks (1 BP per rack, WarpShips only, carries 1 SystemShip per rack)

Example: build set pd=7 b=3 s=2 t=1 m=6

Note: Warp Generator costs 5 BP (automatically added for WarpShips)

build commit *ship_code*, **bc** *ship_code*

Finalize and add draft ship to fleet (deducts BP)

build cancel *ship_code*, **bx** *ship_code*

Cancel draft ship (no BP deducted)

Deployment Commands

deploy *ship_code location*

Deploy newly built ship to specified base star hex

First deployment by either player claims that side of the map (3 base stars). Subsequent deployments must be to the same side.

Movement Commands

move *ship_code destination [via...]*

m *ship_code destination [via...]* Move WarpShip along path to destination

Destination: Star system name or hex ID

Via: Optional intermediate waypoints

Movement costs 1 PD per hex or per warpline jump. Ships must stop at star hexes occupied by enemy ships (combat triggered).

pick *warpship systemship*

Load SystemShip onto WarpShip (costs 1 PD during movement)

drop *warpship systemship*

Unload SystemShip from WarpShip (costs 1 PD during movement)

Combat Commands

combat, c

Show active combat status at all contested hexes

combat order *ship tactic target allocation*

co *ship tactic target allocation* Issue combat order for one of your ships

Tactics: attack (a), dodge (d), retreat (r)

Target: Enemy ship code

Allocation: Power distribution (e.g., pd=5 b=3 s=2)

Example: co W3 attack S25 pd=5 b=3 s=2

For missiles, specify tube count and drive settings:

co S25 dodge W3 pd=5 t=1 m=3

(missile drive setting: 3)

combat commit, cc

Finalize all combat orders (locks them in)

combat cancel, cx

Cancel all pending combat orders

combat drafts, cd

Show your pending combat orders

combat apply ship damage_allocation**ca ship damage_allocation** Assign damage to ship attributes after combat resolution**Example:** ca W3 pd=2 b=1 s=2

Distributes 5 total damage: 2 to PD, 1 to Beam, 2 to Screen

Repair Commands**repair ship_code attributes**

Repair damaged ship at base star hex (1 BP per attribute point)

Example: repair W3 pd=2 b=1

Costs 3 BP total. Ship must start turn at your base hex.

resupply ship_code m=N

Resupply missiles (1 BP per 3 missiles, up to original capacity)

COMBAT RESOLUTION**Combat Results Table**

Outcomes determined by comparing drive allocations and tactics:

Drive Difference = Attacker Drive - Defender Drive

Results: Miss, Hit, Hit+1, Hit+2, or Escapes (for Retreat tactic)

Power Allocation

Each combat order specifies how PD is allocated:

- **Drive:** Maneuverability (affects CRT outcomes)
- **Beam:** Weapon power (up to ship's beam maximum)
- **Screen:** Defense (absorbs hits = power + tech level)
- **Tubes:** Number of missiles to fire (costs 1 PD per tube)

Constraint: Drive + Beam + Screen + Tubes ≤ Current PD

Damage Application

1. Calculate hits from all weapons that hit
2. Subtract Screen power + tech level (if Screen powered)
3. Apply remaining "effective hits" to ship attributes (owner chooses where)
4. Ship destroyed when all attributes except Warp Generator reduced to 0

Special Combat Rules

- Beams and Screens can be used together
- Missiles cannot be used same round as Beams/Screens
- Missiles always use Attack tactic with their own drive setting
- SystemShips cannot Retreat (must be carried by WarpShip)
- Combat ends when: all ships of one side destroyed/retreated, or 3 consecutive rounds with no unabsorbed damage (attacker must withdraw)

MOVEMENT RULES

1. WarpShips must stop at star hexes occupied by enemy ships

2. WarpShips may move through space hexes with enemy ships
3. WarpLines are treated as space hexes for movement
4. Ships with PD=0 cannot move
5. PD used for movement is not expended (full PD available for combat)
6. Cannot move onto enemy base star hex during turn 1

VICTORY CONDITIONS

Dominance Victory

Accumulate 3 victory points by occupying enemy base stars at start of your turn

Elimination Victory

Destroy all enemy ships

Draw Neither player has effective ships remaining

BUILD POINT COSTS

Attribute	Cost
Power/Drive (PD)	1 BP per unit
Warp Generator (WG)	5 BP
Beam (B)	1 BP per unit
Screen (S)	1 BP per unit
Tube (T)	1 BP per tube
Missiles (M)	1 BP per 3 missiles
SystemShip Rack (SR)	1 BP per rack

EXAMPLES

Building a WarpShip

- > **build new** W2 Destroyer
- > **build set** pd=5 b=3 s=2 t=1 m=3
- > **build commit**

The same commands with aliased versions:

- > **bn** w Falcon
- > **bs** pd=5 b=3 s=2 t=1 m=3
- > **bc**

Note: If the player doesn't care, the game will generate a numeric ID. Entering W2 is not necessary -- unless you want your ship numberd W2.

Total cost: $5+3+2+1+1+5(WG) = 17$ BP

Tech level 0 (if built turns 1-4)

Building a SystemShip Fighter

- > **build new** S55 Fighter
- > **build set** pd=1 t=1 m=6
- > **build commit**

Or use:

- > **bn** S55 Avenger
- > **bs** pd=1 t=1 m=6
- > **bc**

Total cost: $1+1+2 = 4$ BP

No Warp Generator (SystemShip)

Movement Example

> **move** W6 Tavon Xylen

Or the short hand:

> **m** W6 Tavon Xylen

W6 moves from current hex to Tavon (1 PD), along warpline to next star (1 PD), then 3 hexes to Xylen (3 PD) = 5 PD total

Combat Order Example

> **combat order** W3 **attack** S25 d=0 b=3 s=2

Or use with same result:

> **co** W3 **a** S25 d=0 b=3 s=2

W3 attacks S25, allocating 0 to drive, 3 to beam, 2 to screen (requires PD¥5)

Missile Fire Example

> **combat order** S25 **dodge** W3 d=4 t=1 m=1

> (missile drive setting: 3)

Or use:

> **co** S25 **d** W3 d=4 t=1 m=1

> (missile drive setting: 3)

S25 dodges while firing 1 missile at W3 with drive=3 (requires PD¥5)

CONSTRAINTS AND INVARIANTS

- No randomness - all outcomes deterministic
- No hidden information - all game state is public
- Rules do not change mid-game
- Equal inputs yield equal outcomes
- Ships retain original tech level permanently
- Warp Generators never take damage (ship destroyed when all else is 0)
- SystemShips cannot have Warp Generators or SystemShip Racks
- WarpShips cannot be carried in SystemShip Racks

REPAIR AND RESUPPLY

- Ship must start turn at your base star hex
- SystemShips can be repaired/resupplied while loaded on WarpShip
- 1 BP repairs 1 unit of any attribute
- Cannot repair beyond original built capacity
- Tech level never changes
- 1 BP resupplies up to 3 missiles
- Cannot resupply beyond original missile capacity

SYSTEMSHIP PICKUP/DROP**During Movement**

- Costs 1 PD per SystemShip picked up or dropped

- Can be done at any star hex during movement

During Combat

- Requires: Drive=0, Screen=0, Dodge or Retreat tactic
- Only 1 SystemShip per combat round
- May fire Beam (but not Missiles) while picking up/dropping
- Dropped SystemShips cannot fire that round
- Picked up SystemShips cannot fire but may power Screens

After Combat

- Free rearrangement of SystemShips at star hexes
- No movement cost
- Active player only

FILES

~/kh/saves/

Directory containing saved game files

~/kh/lastgame

Automatically saved game when quitting

SEE ALSO

RULES.md(5), **COMBAT_RULES.md(5)**, **DESIGN.md(5)**

NOTES

- All ship records must be maintained by players
- Records are shown to opponent after (not during) game
- Track BP received and expended
- Combat orders are written simultaneously and revealed together
- Fractions of BP from missile resupply are not saved

THE MILIEU CODEX

The MILIEU CODEX of Stars is the compendium of known space, containing detailed information about each star system. Knowledge levels range from Unknown (name only) to Intimate (complete secrets).

Systems are organized by territory:

- **Western Alliance:** ARVEN (capital), BELIX, CAYRU
- **Eastern Powers:** ZAREK, ASTREX, BRION
- **Contested Reach:** KORAL, QUELL, SYDRA, VARYN, WEXAR, XYLEN, TAVON, and others

Each system entry includes celestial bodies, notable moons, resources, populations, infrastructure, and anomalies.

RESOURCES

Eight types of raw materials can be extracted and traded:

Resource	Abbrev	Primary Uses

FERROUS	Fe	Hull construction
RARE_EARTH	Re	Electronics, screens
RADIOACTIVE	Rd	Power, weapons
CRYSTALLINE	Cr	Beams, warp drives
VOLATILE	Vo	Fuel, missiles
WATER	H2O	Life support
ORGANIC	Or	Food, medicine
EXOTIC	Ex	Special technology

MARKET DYNAMICS

The commodity market fluctuates based on supply and demand:

- Prices update each round after both players complete turns
- Buying increases demand, pushing prices up
- Selling increases supply, pushing prices down
- Random market forces also affect prices
- Prices clamped to 50%-200% of base value

Use **market** to view current prices and trends, then **trade buy/sell** to act.

FACILITIES

Facilities at star systems provide strategic advantages to the controlling player. Control is established by occupying a system with no enemy ships for 2 consecutive turns.

Facility	Effect
SHIPYARD	Build new ships
REPAIR_DOCK	Repair damaged ships away from base
REFINERY	Fabrication bonuses
TRADE_HUB	+5 CR income per turn
FORTRESS	Combat defensive bonus

Use **repair** at captured REPAIR_DOCKs or SHIPYARDs to repair ships in the field.

AI AGENT PROGRAMMING

Kepler's Horizon supports single-player mode through an AI opponent whose behavior is defined in Kepler Script (KHS), a functional programming language tailored for tactical decision-making. The AI agent replaces one human player and executes commands through the same game systems, respecting all rules and inhibits.

Overview

KHS scripts define AI strategy through a series of decision functions called during each phase of the AI's turn. The server loads */opt/kepler/ai/strategy.khs* at startup and can reload it without restart via **kill -HUP** signal.

Language Characteristics

KHS is a functional hybrid language combining immutability, pattern matching, and first-class functions with familiar C-like syntax:

- Variables immutable by default (**const**)
- Pattern matching via **match** expressions
- Arrow functions for concise definitions
- List comprehensions and pipe operators

- C-like braces and semicolons (no whitespace sensitivity)
- Strong typing with inference

Reserved Words

Keyword	Purpose	Example
const	Immutable binding	const x = 5;
if	Conditional	if (x > 0) return true;
else	Alternative	else return false;
match	Pattern matching	match (tactic) { ... }
when	Match guard	when ratio < 0.3
return	Function result	return Attack;
for	List iteration	for ship in fleet
where	Filter condition	where ship.pd > 0
select	Map transform	select ship.location

Built-in Types

Type	Description
int	Integer numbers
float	Floating point
bool	true or false
string	Text strings
Ship	Ship entity with attributes
HexID	Map location identifier
GameState	Complete game state
Tactic	Attack, Dodge, or Retreat
ShipDesign	Ship specification for building
MovePlan	Movement instructions
PowerAllocation	Combat power distribution
DamageAssignment	Damage application

Core Decision Functions

Every KHS strategy must implement these functions:

buildShip(credits: int, game: GameState) -> Option<ShipDesign>

Decide whether and what to build given available credits. Returns **None** to skip building, **Some(design)** to build.

planMovement(ship: Ship, game: GameState) -> MovePlan

Determine where to move a ship. Returns path as list of hex IDs.

selectTactic(ship: Ship, enemies: Ship[]) -> Tactic

Choose combat tactic (Attack, Dodge, or Retreat) based on situation.

allocatePower(ship: Ship, tactic: Tactic, enemies: Ship[]) -> PowerAllocation

Distribute ship's PD among drive, beam, screen, and tubes.

assignDamage(ship: Ship, damage: int) -> DamageAssignment

Choose which attributes absorb damage.

Built-in Functions

Function	Description
pathfind(from, to, limit)	Find shortest path (uses MapGraph)
evaluateStrength(ship)	Estimate combat effectiveness
min(a, b)	Minimum value
max(a, b)	Maximum value
floor(x)	Round down
filter(list, pred)	Select matching elements
map(list, fn)	Transform elements
sum(list)	Total of numeric list
count(list)	List length
first(list)	First element

Practical Example: Ship Building

```
const buildShip = (credits: int, game: GameState) =>
  Option<ShipDesign> {

    if (credits < 15) return None;

    // Check if we need carriers
    const carriers = game.myShips
      |> filter(s => s.systemRacks > 0)
      |> count();

    if (carriers < 2 && credits >= 20) {
      return Some(buildCarrier(credits));
    } else {
      return Some(buildAttacker(credits));
    }
  };

const buildAttacker = (credits: int) => ShipDesign {
  const budget = credits - 5; // Warp Generator

  return {
    type: WarpShip,
    name: "Attacker",
    pd: floor(budget * 0.30),
    beam: floor(budget * 0.45),
    screen: floor(budget * 0.15),
    tubes: floor(budget * 0.10),
    missiles: floor(budget * 0.10) * 3,
    systemRacks: 0
  };
};
```

Practical Example: Combat Tactics

```
const selectTactic = (ship: Ship, enemies: Ship[]) => Tactic {
  const myStrength = evaluateStrength(ship);
  const enemyStrength = enemies
    |> map(evaluateStrength)
    |> sum();
  const ratio = myStrength / enemyStrength;
```

```

    return match (ratio, ship.isWarpShip) {
      (r, true) when r < 0.30 => Retreat,
      (r, _) when r > 1.50   => Attack,
      _                      => Dodge
    };
  };

const allocatePower = (ship: Ship, tactic: Tactic,
                      enemies: Ship[]) => PowerAllocation {
  return match (tactic) {
    Attack => {
      pd: ship.currentPD,
      beam: min(ship.beamMax, ship.currentPD / 2),
      screen: 0,
      tubes: ship.hasMissiles() ? min(2, ship.tubes) : 0,
      missiles: ship.hasMissiles()
        ? repeat(ship.pdMax, 2)
        : []
    },
    Dodge => {
      pd: ship.currentPD,
      beam: ship.currentPD / 3,
      screen: ship.currentPD / 3,
      tubes: 0,
      missiles: []
    },
    Retreat => {
      pd: ship.currentPD,
      beam: 0,
      screen: 0,
      tubes: 0,
      missiles: []
    }
  };
};

```

Practical Example: SystemShip Tactics

```

const shouldDropSystemShip = (carrier: Ship, systemship: Ship,
                             hexId: HexID, game: GameState) => bool {
  // Garrison friendly bases
  if (game.isMyBaseHex(hexId)) return true;

  // Support attacks on enemy bases
  if (game.isEnemyBaseHex(hexId) &&
      game.myShipsAt(hexId).count() > 1) {
    return true;
  }

  // Preserve systemship if carrier retreating
  const enemies = game.enemyShipsAt(hexId);
  if (shouldRetreat(carrier, enemies)) return true;

  return false;
};

```

```

const shouldPickupSystemShip = (carrier: Ship, systemship: Ship,
                                game: GameState) => bool {
  if (!carrier.hasFreeRack()) return false;

  // Evacuate if enemies approaching
  const threats = game.enemyShipsWithinRange(
    systemship.location, 3
  );

  return threats.count() > 0;
};

```

Ring-0 Strategies

The server ships with three baseline KHS strategies in `/opt/kepler/ai/` demonstrating different playstyles:

aggressive.khs

Offensive focus: 45% beam, 30% PD, 15% screen. Attacks when ratio > 1.5, retreats only when ratio < 0.3. Rushes enemy bases early.

defensive.khs

Defensive focus: 20% beam, 40% screen, 30% PD. Prefers Dodge tactic, retreats at ratio < 0.5. Garrisons own bases first.

balanced.khs

Mixed approach: 35% beam, 30% PD, 20% screen. Adapts tactics based on local strength ratios. Builds mix of carriers and attackers.

Reloading Strategy

Edit the active strategy file, then signal the server:

```

# Edit strategy
vim /opt/kepler/ai/strategy.khs

# Reload without restart
kill -HUP $(pidof khserver)

# Verify in logs
tail -f /var/log/kepler/server.log
# [AI] Reloading: /opt/kepler/ai/strategy.khs
# [AI] â Reload successful

```

Changes take effect on the AI's next turn. Failed reloads keep the previous strategy active.

Best Practices

1. Use pattern matching for decision trees rather than nested if-else
2. Prefer pure functions - same inputs always produce same outputs
3. Leverage pipe operator (`|>`) for readable data transformations
4. Keep functions focused - one decision per function
5. Test strategies against each other by swapping active script
6. Balance offense and defense - pure strategies are exploitable

7. Remember SystemShips are force multipliers (no WG cost)

MONITOR MODE

The server may accept commands from the terminal in lieu of player access from a web server. The UI is limited, and each command must be prefixed with notation for which player is issuing the command.

This feature is TBD.

BUGS

Report bugs to the Kepler's Horizon issue tracker.

AUTHORS

Game design and implementation by sibomots.

COPYRIGHT

Copyright (c) 2025 sibomots. Licensed under BSD 3-Clause License.

GAME DESIGN PHILOSOPHY

Kepler's Horizon is intentionally minimal. Strategic depth emerges from:

- Constrained movement
- Simultaneous decision-making
- Irreversible consequences

The rules favor clarity over complexity.